

# **MOST ASKED MICROSERVICES INTERVIEW QUESTIONS**

in Java Interviews (2026)



@java\_in.depth

# Microservices Interview Questions Top 15 Most Asked Questions (Beginner to Advanced) Java | Spring Boot Edition

Save this post, if you're preparing for Java Interviews.

# 1. WHAT ARE MICROSERVICES?

Microservices is an architectural style where a large application is built as a collection of small, independent, loosely coupled services. Each service focuses on one business capability, runs in its own process, and communicates via APIs (usually REST or messaging).

**Key Takeaway:** Independent deployment & scalability.

## 2. MONOLITHIC VS MICROSERVICES ARCHITECTURE?

**Monolith:** Single large codebase, single database, everything tightly coupled.

**Microservices:** Multiple small services, each with its own database, loosely coupled.

**Pros of Microservices:** Independent scaling, technology freedom, faster deployment.

**Cons:** Complexity in communication, distributed tracing, and data consistency.

**When to choose?** Start with monolith, move to microservices when the app grows big.

### **3. WHAT ARE THE MAIN ADVANTAGES OF MICROSERVICES?**

- Independent deployment & scaling
- Technology diversity (different services can use different tech)
- Fault isolation (one service down  $\neq$  whole app down)
- Faster development & easier maintenance
- Better team ownership (each team owns one service)

## 4. WHAT IS AN API GATEWAY IN MICROSERVICES?

API Gateway is the single entry point for all clients (web, mobile, etc.). It handles: routing, load balancing, authentication, rate limiting, and request transformation.

**Popular tools:** Spring Cloud Gateway, Zuul, Kong, AWS API Gateway.

**Real use:** Hides internal service structure from clients.

# 5. HOW DO MICROSERVICES COMMUNICATE WITH EACH OTHER?

**Two main ways:**

1. Synchronous → REST (using RestTemplate/WebClient) or gRPC
2. Asynchronous → Message brokers like Kafka, RabbitMQ, ActiveMQ

Best Practice: Prefer async communication for loose coupling.



## 6. WHAT IS SERVICE DISCOVERY?

In microservices, services are dynamic (they scale up/down). Service Discovery helps services find each other automatically.

### **Types:**

- Client-side (Netflix Eureka)
- Server-side (with API Gateway)

Spring Cloud Netflix Eureka is very popular in Java.



## 7. WHAT IS CIRCUIT BREAKER PATTERN?

It prevents cascading failures. If one service is slow or down, the circuit breaker “opens” and stops calling it. Instead, it returns a fallback response.

Popular library in Java: Resilience4j (recommended now) or Hystrix.

**Real scenario:** When payment service is down, show “Try again later” instead of crashing the order service.

## 8. WHAT IS DISTRIBUTED TRACING?

In microservices, one user request can travel across 10+ services. Distributed Tracing tracks the full request journey with unique Trace ID & Span ID.

**Popular tool:** Spring Cloud Sleuth + Zipkin or OpenTelemetry + Jaeger.

It helps debug performance issues easily.

## 9. HOW DO YOU HANDLE DATA CONSISTENCY IN MICROSERVICES?

Each service has its own database (Database per Service pattern).

### **We use:**

- Saga Pattern (orchestration/choreography)
- Event Sourcing + CQRS
- 2-Phase Commit (avoid if possible, it's slow)

Key: Prefer eventual consistency over strong consistency.

## 10. WHAT IS SAGA PATTERN?

Saga manages long-running transactions across multiple microservices. It breaks one big transaction into multiple local transactions + compensating actions (rollback).

### **Two types:**

- Choreography (event-based)
- Orchestration (central coordinator)

# 11. EXPLAIN CLIENT-SIDE VS SERVER-SIDE LOAD BALANCING

**Client-side:** Client (or API Gateway) decides which instance to call (e.g., Ribbon + Eureka).

**Server-side:** Load balancer (like Nginx or cloud LB) sits in front and routes traffic.

**Modern trend:** Use Spring Cloud LoadBalancer + API Gateway.

# 12. HOW DO YOU SECURE MICROSERVICES?

## Common practices:

- OAuth2 + JWT for authentication & authorization
- API Gateway as security layer
- mTLS (mutual TLS) for service-to-service
- Role-based access with Spring Security
- Rate limiting & IP whitelisting

## 13. WHAT IS CONFIG SERVER?

Centralized configuration management using Spring Cloud Config Server (Git backend). All services pull config at runtime.

## 14. WHAT IS CONTAINERIZATION & ORCHESTRATION?

Use **Docker** to containerize services + Kubernetes for deployment, scaling, and management.



# 15. NAME SOME COMMON MICROSERVICES DESIGN PATTERNS

- API Gateway, Circuit Breaker, Saga, CQRS, Event Sourcing, BFF (Backend for Frontend), Strangler Fig Pattern.

Save this post if you're preparing for Java Interviews.



**WANT THIS PDF?**

GET PDF ON TELEGRAM!  
LINK IN BIO.